Where complexity concentrates
Coupling divergence as a placement outlier that detects god classes across 11 languages

Daniel Sagenschneider
OfficeFloor
daniel@officefloor.net

October 2026

Abstract

A companion study [1] argued that a problem carries a fixed amount of complexity, so architecture is where that complexity sits rather than how much of it there is. A healthy system spreads it. An unhealthy one lets it pile up in one place. If the amount is conserved, a spike in one location is pure concentration, and concentration is the thing to avoid. This paper gives the instrument that sees the spike. For every function we read the arguments passed at each call to the project's own code as flow over a control edge. A function that pushes out more flow than it takes in manufactures coupling. We call the surplus its divergence. A god is the dominant outlier in divergence, a place where flow is created rather than passed along. The measure needs no history and no thresholds. It is a relative outlier at a single snapshot. We test detection on a seeded, stratified sample of 98 open-source repositories across 11 languages, 597,117 functions in all, against two independent textbook references, per-file weighted methods per class and the full Lanza and Marinescu God Class strategy run by PMD. Divergence outliers are enriched for textbook gods by a mean factor of 2.81 at the class level, with a 95% interval of 2.54 to 3.10 that excludes 1, and by 4.4 at the method level. Every one of the 11 languages has an interval above 1. The effect is a complementary axis. It agrees with the references on most gods and flags a further kind that a foreign-data test misses, the outward orchestrator. All code, the sampler, the frozen population and the per-repository results are public so the study can be rerun.

1 Introduction

The companion study measured sixty accumulating change requests on one endpoint under two architectures and found that the total complexity barely moved while its placement moved a great deal [1]. The reading there follows Tesler and Brooks [2, 3]. A problem has an essential, irreducible amount of complexity, and a solution can only add accidental complexity on top. If the essential part is fixed, the lever an architect actually controls is placement, not quantity. The healthy pattern is an even spread. The failure mode is a pile-up, a single unit that accretes more and more until it becomes the part no one wants to touch. Practitioners already have a name for that unit. It is the god class, or the god method.

Conservation is what makes a pile-up worth detecting. When the amount is fixed, a tall spike in one place is not matched by a saving somewhere else. It is pure concentration. So the diagnostic question is simple to state. Where is the spike. The contribution of this paper is a measure that answers it, and a cross-language study that shows the answer lines up with what people already call a god.

The measure is COUPLING DIVERGENCE. It treats the arguments passed at a call as flow carried over a control edge, and it counts, per function, the flow pushed out against the flow

taken in. A function that pushes out more than it takes manufactures coupling. The surplus is where control flow stops behaving like a network of pipes, which conserve what passes through them, and starts creating flow instead. That point of creation is the concentration the companion study said to avoid. A god is the dominant outlier in it.

Two properties matter. First, the measure reads one checkout and needs no history, because concentration is visible in placement already. It identifies the god from the code as it stands. Second, it is threshold-free. A god is a relative outlier against the rest of the same codebase, found with a modified z-score, not a fixed cutoff. A small service and a large monolith are each judged against themselves.

The claim is identification. On 98 repositories the divergence outlier is enriched for textbook gods across every language we sampled, which Section 4 reports in full.

2 The measure

2.1 Two flows over the same edges

A call is a control edge. The arguments passed at that call are the data that crosses it. We read the two separately, which is the same separation of control flow from data flow that motivates the companion architecture [1]. For a function v, over its calls to the project's own code, let

$$\mathrm{out}(v) = \sum_{\text{internal call sites in } v} (\text{argument count}),$$
$$\mathrm{in}(v) = \sum_{\text{callers of } v} (\text{argument count pushed into } v).$$

Both sums range only over calls whose callee is a function defined somewhere in the tree. A call that leaves the tree into a library or into SQL is not counted. This single rule is what keeps a pure converter or mapper from looking like a god. A method that unpacks a wide object and hands every field to an external framework has a large raw fan-out, but none of it lands on the project's own code, so it carries no divergence.

The divergence of v is the net flow it manufactures,

$$\delta(v) = \max\big(\mathrm{out}(v) - \mathrm{in}(v),\, 0\big).$$

A pipe joint that merely splits incoming flow has out $\approx$ in and near-zero divergence. A joint that takes a little and sends a lot to many places has a large positive divergence. It is a source. Negative values are clamped, because a pure sink is not a god, it is a leaf that others feed.

2.2 Conservation and creation

The picture is deliberate. If control flow were a plumbing network, flow would be conserved at every joint, and divergence would be zero everywhere. Information systems are not plumbing. They create information. A handler reads one identifier and builds a whole object graph out of it, then pushes that graph outward over many calls. That creation is not a defect in itself. It is what the program is for. The defect is when creation concentrates into one unit instead of being spread. So the healthy state is low, even divergence, and the unhealthy state is a spike. The spike is the god. The name fits the mechanism. A god is the place where flow is created rather than passed along.

2.3 From function to god

A god method is the dominant, persistent outlier in δ among the functions that manufacture any flow. We use the Iglewicz and Hoaglin modified z-score [17], built on the median and the

median absolute deviation, with the usual cutoff of 3.5. The median and the deviation are robust, so one extreme value does not drag the scale and hide the next one. There is no absolute threshold, so the test transfers across languages and across project sizes without tuning.

A god class is the same idea one level up. We sum δ over the methods in a file and take the outlier of that per-file total. A file is the language-neutral unit here, since the measure runs on every language the parser handles and not every language has classes. This per-file sum is matched to weighted methods per class, which is also a per-file sum, so the two are compared on the same footing.

2.4 Relation to classical flow metrics

Two design choices separate divergence from the measures nearest to it, and Section 5 places it in the wider landscape. The first choice is the signed difference. The Henry and Kafura information flow [4] weights a procedure by length $\times$ (fan-in $\times$ fan-out)^2. That product is symmetric. A pure source, with fan-in near zero, scores near zero, which is exactly the entry-point god this measure is built to catch. Divergence is a signed difference, not a product, so a source scores high. The second choice is to weight by argument flow and read the result as a within-codebase outlier. Martin's package metrics take the same out-against-in asymmetry and read it as a stability ratio, efferent coupling over total coupling [9]. Divergence counts the arguments that cross each own-code call rather than the bare dependency, keeps the net surplus rather than a ratio, and asks which unit is the outlier rather than where a package sits on a stability line. A search of the literature found no prior god definition built on non-conservation of argument flow.

3 Study design

3.1 Detection, stated as enrichment

The claim under test is that divergence outliers are where the textbook gods are. We state it as enrichment. Within one repository, let the universe be the functions that manufacture any flow. Among them, some are divergence outliers and some are textbook gods. Lift is the god rate among the outliers divided by the god rate over the whole universe. A lift of 1 means the outlier set is no richer in gods than chance. A lift above 1 means it is. Recall is the share of textbook gods the outlier set recovers. We also report a one-sided Fisher exact p-value per repository, so each repository is one honest test on its own two-by-two table.

3.2 Three references, chosen to be hard

We compare against references that do not share the measure's machinery. Weighted methods per class at the file level, with the Lanza and Marinescu cutoff of 47, is a pure size reference computed by `lizard` and blind to dependency injection. Cyclomatic complexity of 14 or more is the method-level size reference. For the Java repositories we run PMD with the full God Class rule, which is the complete Lanza and Marinescu strategy, weighted methods and foreign-data access and low cohesion together. A measure that only restated size would track the weighted methods reference and fail the PMD one. The point of the three is to separate size from the god pattern.

3.3 Sampling, and the conflict of interest it answers

The author builds and tests his own ecosystem, which is a self-selection threat. The answer is to put the choice of population in code and make it auditable, so anyone can reproduce how the sample was drawn, not just the measurement. The sampler queries the GitHub search API through the `gh` client, stratified by language and by popularity tier, with high above 5000 stars,

mid from 1000 to 5000, and low from 200 to 1000. It draws a seeded random k per stratum and writes the repository list together with a manifest that records the seed, the date, every query, the match count per stratum, and the stars and rank of each selection. The author's own owners are excluded by construction, both with a GitHub qualifier and with a post-fetch filter, and the exclusion is recorded in the manifest. A rerun reproduces the method, not an identical list, because GitHub is a moving target. The committed manifest and the per-repository commit hashes pin the exact population that produced these numbers. One seeded draw, with $k = 3$ over 11 languages and 3 tiers, gave the 99 repositories studied here.

3.4 When to stop sampling

The tool reports, from the observed between-repository variance of log-lift, how many repositories a target precision would need. We set the target at a 95% interval half-width of 0.2 on log-lift. The aggregate claim needed 21 repositories at the class level and 30 at the method level. The sample is past both, so the aggregate result is reported at its target precision rather than at whatever size happened to be convenient.

4 Results

Of the 99 sampled repositories, 98 were measured and 1 held no analysable source. The measured set spans 597,117 functions in 70,425 files and contains 12,479 files at or above the weighted-methods god cutoff.

4.1 Aggregate detection

Treating each repository as one observation, the mean log-lift of the class-level divergence outlier against the weighted-methods reference is 2.81 as a lift, with a 95% interval of 2.54 to 3.10. The interval excludes 1. Of the 79 repositories with a usable class-level lift, 57 are individually significant at a Fisher p below 0.05. At the method level, against cyclomatic complexity of 14 or more, the mean lift is 4.4 over 76 repositories, with an interval of 3.88 to 4.98, again excluding 1, and 50 repositories individually significant.

Against the harder reference, PMD's full God Class strategy on the Java repositories, the mean lift is 3.20 over 7 repositories with an interval of 2.54 to 4.05. The measure recovers the complete, cohesion-aware strategy about as well as it recovers the pure size cutoff, which is the result that separates it from a restatement of size.

4.2 Every language, not just the pool

Table 1 breaks the class-level lift down by language. Every one of the 11 languages has a 95% interval whose lower bound is above 1. This is not a case of one language carrying the pool. The direction holds everywhere. The intervals differ in width, and five languages are wide because they drew few repositories with high between-repository variance. We therefore report direction and significance per language, which hold, and we do not rank the languages against one another by effect size, which their intervals do not support.

4.3 Popularity and kind

Breaking the class-level lift by popularity tier gives 2.88 for high, 3.18 for mid, and 3.53 for low, so the effect is present across the quality proxy and does not depend on it. A post-hoc split into application and library, which the sampler does not control and which we report rather than claim, gives 2.93 for 60 applications and 3.72 for 38 libraries.

Table 1: Class-level enrichment by language. Lift is the mean over repositories, with a 95% interval on log-lift. Every interval excludes 1. "needed" is the repository count for an interval half-width of 0.2 on log-lift, the target precision. Five languages sit below it and are reported for direction only.

language	repos	lift	95% interval	needed
c	14	2.11	1.77–2.50	11
c#	8	3.64	2.34–5.67	40
c++	3	2.25	1.42–3.55	16
go	7	4.54	2.90–7.13	36
java	8	3.22	2.72–3.82	6
js	5	2.79	1.83–4.27	23
php	7	3.00	2.44–3.69	8
python	7	2.08	1.69–2.55	8
ruby	4	2.46	1.44–4.19	29
rust	8	2.25	1.89–2.69	7
ts	8	3.62	2.92–4.50	10
all	79	2.81	2.54–3.10	21

4.4 A complementary axis, not a restatement of size

The measure is threshold-free, so it is not a size cutoff in disguise. The reference comparison shows the same thing from the other side. Among the files that weighted methods calls a god but PMD does not, that is, files that are large but do not show the foreign-data and low-cohesion pattern, there are 361 across the Java repositories, and divergence flags 52 percent of them. It does not avoid these files and it does not blindly follow size either. It flags the ones that manufacture outward flow and leaves the ones that are merely large and self-contained. That is the complementary kind of god, the outward orchestrator, which a foreign-data test is built to miss and which divergence is built to catch.

5 Related work

Coupling divergence sits at the meeting of several older lines of work. It is new as a measure, but it is not isolated, and the honest account names what it is closest to as well as what it differs from.

Detecting the god, and the blob it names. The unit this paper detects has a long pedigree. Fowler lists the Large Class and the companion smells of Feature Envy and Shotgun Surgery, where one unit reaches across many others [8]. Lanza and Marinescu formalise the God Class as a detection strategy, a thresholded conjunction of weighted methods, access to foreign data, and low cohesion [5], built on Marinescu's earlier work on metric-based detection strategies for design flaws [6]. Moha and colleagues specify and detect code and design smells, the Blob among them, with the DECOR method [7]. These define what a god is. We use two of them as the references our outlier is tested against, and the result in Section 4 is that divergence agrees with them on most gods and adds a further kind.

Coupling direction and information flow. The measures closest to divergence come from this line. Chidamber and Kemerer count coupling between objects [16]. Henry and Kafura weight a procedure by its fan-in and fan-out [4]. Martin reads the direction of coupling, efferent against afferent, as package instability [9]. Divergence is most similar to that last idea. It takes the same out-against-in asymmetry, but it weights each edge by the arguments that cross

it, keeps the net surplus rather than a ratio, restricts the count to the project's own code, and flags the outlier rather than placing a package on a stability line. Where Martin's metric guides a design toward a stable dependency structure, divergence points at the single unit that manufactures the most coupling.

Where complexity concentrates. A separate line asks the same question this paper does, at the architecture level. MacCormack, Rusnak and Baldwin use the design structure matrix to measure propagation cost and to find the core, a cluster of files that are reachable from one another and carry most of the system's coupling [10]. Mo, Cai, Kazman and Xiao define hotspot patterns and detect them from the design structure matrix and history together [11]. Treating a program as a graph, the study of software as a complex network finds that a few hub nodes carry a disproportionate share of the edges [12, 13]. Divergence locates the same concentration, but as a local per-node snapshot measure. It needs no design structure matrix, no history, and no whole-system reachability computation, and it runs on any language the parser reads. The companion study makes the matching architectural argument, that the amount of complexity is conserved and only its placement varies [1].

Relative thresholds rather than fixed ones. The decision to call a god a within-codebase outlier, rather than a value past a fixed cutoff, follows a methodological line. Alves, Ypma and Visser derive metric thresholds from benchmark data rather than setting them by hand [14]. Oliveira, Valente and Lima extract relative thresholds that a system meets for most of its code [15]. Our modified z-score [17] is in the same spirit, judging each unit against the distribution of its own codebase, so the test transfers across languages and sizes without tuning.

6 Scope and limitations

The sample is popularity-stratified open-source code. The claim generalises to widely used open-source projects, not to all software. The application and library split is a heuristic, not a controlled factor. The per-language intervals for five languages are wide, as Table 1 records, so we report direction and significance for those languages and not a precise effect size. The references are themselves imperfect oracles. Weighted methods is a size cutoff, and the Lanza and Marinescu strategy carries its own thresholds, so agreement is evidence that the outlier marks the same units practitioners already call gods, not a claim of ground truth.

7 Reproducibility and competing interests

The measure, the sampler, the study runner and the tests are public. The frozen repository list and the sampling manifest pin the population, and the per-repository results table pins each measurement to a repository and a commit, so any row can be checked by checking out that commit and remeasuring it. The ancillary package bundles the results table, the frame and a self-contained description of every column. The commands that produced the study are recorded with the code.

The competing interest is stated directly. The author develops the ecosystem whose philosophy the companion study argues for. The response is openness. Everything needed to reproduce the sampling, the measurement and the statistics is public, and the author's own repositories are excluded from the population by construction. The numbers here stand or fall on an independent rerun, which the released code makes possible.

8 Conclusion

If complexity is conserved and only its placement is free, then the defect to find is concentration, and concentration is a spike. Coupling divergence measures the spike as net flow manufactured over a program's own control edges, and marks the god as its outlier. Across 98 repositories in 11 languages the outlier is where the textbook gods are, by a factor of 2.81 at the class level with every language above chance. The measure is threshold-free, needs no history, and catches a kind of god that a foreign-data test misses. It is an instrument for the placement the companion study said was the real variable.

References

[1] D. Sagenschneider. *Conserved amount, negotiable placement. Prompting perturbs one architecture, an AI reviewer both.* arXiv:2610.14130, 2026.

[2] L. Tesler. *The law of conservation of complexity.* As discussed in B. Moggridge, *Designing Interactions*, MIT Press, 2007.

[3] F. P. Brooks. *No Silver Bullet. Essence and accidents of software engineering.* Computer, 20(4), 1987.

[4] S. Henry and D. Kafura. *Software structure metrics based on information flow.* IEEE Transactions on Software Engineering, SE-7(5), 1981.

[5] M. Lanza and R. Marinescu. *Object-Oriented Metrics in Practice.* Springer, 2006.

[6] R. Marinescu. *Detection strategies. Metrics-based rules for detecting design flaws.* International Conference on Software Maintenance (ICSM), 2004.

[7] N. Moha, Y.-G. Guéhéneuc, L. Duchien, and A.-F. Le Meur. *DECOR. A method for the specification and detection of code and design smells.* IEEE Transactions on Software Engineering, 36(1), 2010.

[8] M. Fowler. *Refactoring. Improving the Design of Existing Code.* Addison-Wesley, 1999.

[9] R. C. Martin. *OO design quality metrics. An analysis of dependencies.* 1994. Also in *Clean Architecture*, Prentice Hall, 2017.

[10] A. MacCormack, J. Rusnak, and C. Y. Baldwin. *Exploring the structure of complex software designs. An empirical study of open source and proprietary code.* Management Science, 52(7), 2006.

[11] R. Mo, Y. Cai, R. Kazman, and L. Xiao. *Hotspot patterns. The formal definition and automatic detection of architecture smells.* Working IEEE/IFIP Conference on Software Architecture (WICSA), 2015.

[12] C. R. Myers. *Software systems as complex networks. Structure, function, and evolvability.* Physical Review E, 68(4), 2003.

[13] S. Valverde and R. V. Solé. *Hierarchical small worlds in software architecture.* Dynamics of Continuous, Discrete and Impulsive Systems, Series B, 14, pp. 1–11, 2007. Preprint arXiv:cond-mat/0307278.

[14] T. L. Alves, C. Ypma, and J. Visser. *Deriving metric thresholds from benchmark data.* International Conference on Software Maintenance (ICSM), 2010.

[15] P. Oliveira, M. T. Valente, and F. P. Lima. *Extracting relative thresholds for source code metrics.* Conference on Software Maintenance, Reengineering and Reverse Engineering (CSMR-WCRE), 2014.

[16] S. R. Chidamber and C. F. Kemerer. *A metrics suite for object oriented design.* IEEE Transactions on Software Engineering, 20(6), 1994.

[17] B. Iglewicz and D. C. Hoaglin. *How to Detect and Handle Outliers.* ASQC Quality Press, 1993.

[18] T. Yin. *Lizard. An extensible cyclomatic complexity analyzer for many languages.* `https://github.com/terryyin/lizard`.

[19] PMD contributors. *PMD. An extensible source code analyzer.* `https://pmd.github.io`.